

Python E-Course

Module 12 — Contours & Shape Analysis

Detailed Notes

Module Overview

Level: Segmentation

Duration: ~1 week

Prerequisites: Modules 1-11

What You'll Learn

- Find and draw contours.
- Use hierarchy modes (RETR_EXTERNAL, RETR_TREE).
- Compute shape descriptors: area, perimeter, moments, aspect ratio, extent, solidity.
- Compute bounding boxes and convex hulls.

Lessons

#	Lesson	Duration
1	Finding Contours	25 min
2	Drawing & Hierarchy	25 min
3	Moments & Centroid	25 min
4	Bounding Boxes & Hull	25 min
5	Shape Descriptors	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 13: Features

Lesson 1: Finding Contours

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Run `cv2.findContours` on a binary mask.
- Understand what a contour is structurally.
- Filter contours by area.

Prerequisites

- Modules 1-11

1. Why This Matters

Contours are the polygonal outlines of connected regions. Once you have them, you can count objects, measure shapes, recognise letters, filter by size — they're the bridge between pixel-level segmentation and object-level reasoning.

2. Concept

2.1 What a contour is

A NumPy array of (x, y) points along an object's boundary, shape (N, 1, 2). OpenCV returns a Python list of these arrays — one per detected region.

2.2 The function

```
contours, hierarchy = cv2.findContours(mask, mode, method)
```

- `mask` — binary uint8 (0 or non-zero).
- `mode` — retrieval mode (next lesson). `cv2.RETR_EXTERNAL` for just outer contours; `cv2.RETR_TREE` for full hierarchy.
- `method` — approximation. `cv2.CHAIN_APPROX_SIMPLE` (compress straight runs to endpoints) or `cv2.CHAIN_APPROX_NONE` (every point).

2.3 Output

`contours`: list of arrays of points. `hierarchy`: parent/child/sibling info per contour (Lesson 2).

2.4 Source must be binary

Threshold, edge-detect, or mask first. `cv2.findContours` on a grayscale image is undefined.

2.5 Filtering by area

```
contours = [c for c in contours if cv2.contourArea(c) > 100]
```

Drops tiny noise blobs. Almost always the first thing you do after finding.

2.6 In-place modification (versions ≤ 4.0)

Older OpenCV versions modified the input mask. Modern versions (4.x) don't. Still good practice to pass `mask.copy()` to avoid surprises.

3. Code Examples

Example 1: Find contours of binarised coins

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
print("contours found:", len(contours))
print("first contour shape:", contours[0].shape)
print("sample points (first 3):", contours[0][:3, 0])
```

Expected Output:

```
contours found: ~25
first contour shape: (~50, 1, 2)
sample points (first 3): [[210  19] [209  20] [...]]
```

Example 2: Filter by area

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
big = [c for c in contours if cv2.contourArea(c) > 500]
print("all:", len(contours), " big (area > 500):", len(big))
```

Expected Output: All ~25 but only the genuine coins (e.g. ~20) survive after filtering noise specks.

Example 3: Draw on the image

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
cv2.imwrite("coins_contours.png", img)
```

Expected Output: Coin image with green outlines around each coin.

Example 4: CHAIN_APPROX comparison

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

```

c_none, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
c_simple, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
print("approx NONE points (largest): ", max(len(c) for c in c_none))
print("approx SIMPLE points (largest):", max(len(c) for c in c_simple))

```

Expected Output: NONE returns ~hundreds of points per coin (every boundary pixel); SIMPLE returns far fewer (just the endpoints of straight runs).

4. Parameter Sensitivity

Param	Effect
mode	EXTERNAL → only outer; TREE → outer + holes + nesting
method	NONE → full points; SIMPLE → compressed
area filter	bigger threshold = fewer contours

5. Common Mistakes

Mistake	What Happens	Fix
Pass grayscale to findContours	Undefined	Threshold first
Forget to filter tiny contours	Hundreds of bogus blobs	Filter by area early
Treat contours as one array	TypeError	It's a list of arrays
Pass hierarchy is None confusingly	Just ignore if using EXTERNAL	Use _

6. Practice Tasks

- Otsu-threshold coins(); find contours; print count.
- Filter to area > 500; count remaining.
- Draw all contours on the color version; save.

7. Key Takeaways

- cv2.findContours(mask, mode, method) returns (contours, hierarchy).
- Each contour is an (N, 1, 2) array of points.
- Filter by cv2.contourArea to drop noise.
- Use RETR_EXTERNAL + CHAIN_APPROX_SIMPLE as default.

8. What's Next

Drawing and hierarchy — how to render contours and understand parent/child relationships.

Lesson 2: Drawing & Hierarchy

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Use `cv2.drawContours` for outlining and filling.
- Read the 4-tuple hierarchy structure.
- Compare `RETR_EXTERNAL`, `RETR_LIST`, `RETR_CCOMP`, `RETR_TREE`.

Prerequisites

- Module 12 Lesson 1

1. Why This Matters

Without hierarchy you can't tell "outer object" from "hole inside object". Most real images have nested shapes — letters with holes, donuts, gears.

2. Concept

2.1 `drawContours`

```
cv2.drawContours(img, contours, contourIdx, color, thickness)
```

- `contourIdx = -1` → draw all.
- `thickness = -1` (or `cv2.FILLED`) → fill the contour.

2.2 Retrieval modes

Mode	Returns
<code>RETR_EXTERNAL</code>	only the outermost contours
<code>RETR_LIST</code>	all, no hierarchy info
<code>RETR_CCOMP</code>	2-level hierarchy (outer + holes)
<code>RETR_TREE</code>	full nested hierarchy

For "find each object once, regardless of holes inside": `EXTERNAL`. For "find objects and their holes": `CCOMP` or `TREE`.

2.3 Hierarchy format

```
hierarchy.shape    # (1, N, 4)
# Per contour: [Next, Previous, First_Child, Parent]
```

- `Next`: next contour at the same hierarchy level.
- `Previous`: previous at the same level.
- `First_Child`: first hole inside this contour.
- `Parent`: enclosing contour.

-1 means "no such relationship".

2.4 Practical hierarchy use

Find outer contours that have at least one child (i.e., objects with holes):

```
outer_with_holes = []
for i, h in enumerate(hierarchy[0]):
    if h[3] == -1 and h[2] != -1:    # no parent + has child
        outer_with_holes.append(contours[i])
```

2.5 When you don't need hierarchy

Most everyday work uses RETR_EXTERNAL and ignores hierarchy. Reach for TREE only when nested structure matters.

3. Code Examples

Example 1: Draw all contours

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
cv2.imwrite("coins_drawn.png", img)
```

Expected Output: Coins outlined in green.

Example 2: Draw a single contour, filled

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
largest = max(contours, key=cv2.contourArea)

img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
cv2.drawContours(img, [largest], 0, (0, 0, 255), thickness=cv2.FILLED)
cv2.imwrite("largest_filled.png", img)
print("largest area:", cv2.contourArea(largest))
```

Expected Output: Only the biggest coin filled red; others untouched.

Example 3: Hierarchy with TREE

```
import cv2, numpy as np

# Build a shape with holes
img = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(img, (50, 50), (350, 350), 255, -1)
cv2.circle(img, (150, 200), 30, 0, -1)
cv2.circle(img, (250, 200), 30, 0, -1)
```

```

contours, hierarchy = cv2.findContours(img, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
print("contours:", len(contours))
print("hierarchy:\n", hierarchy[0])

```

Expected Output: 3 contours (outer square + 2 holes). Hierarchy shows outer has children, holes have parent = 0.

Example 4: Distinguish outer vs holes

```

import cv2, numpy as np

img = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(img, (50, 50), (350, 350), 255, -1)
cv2.circle(img, (200, 200), 50, 0, -1)

contours, hierarchy = cv2.findContours(img, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_SIMPLE)
disp = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
for i, c in enumerate(contours):
    parent = hierarchy[0][i][3]
    color = (0, 255, 0) if parent == -1 else (0, 0, 255)
    cv2.drawContours(disp, [c], 0, color, 2)
cv2.imwrite("outer_vs_hole.png", disp)

```

Expected Output: Outer rectangle in green, inner hole in red.

4. Parameter Sensitivity

(See L1 for mode/method effects.)

5. Common Mistakes

Mistake	What Happens	Fix
RETR_EXTERNAL but expecting holes	Holes missed	Use CCOMP or TREE
Hierarchy indexing: hierarchy[i]	wrong: it's hierarchy[0][i]	Always hierarchy[0] first
Drawing with non-list contour	TypeError	Wrap single contour: [contour]

6. Practice Tasks

- Draw all contours of binarised coins() in red.
- Fill the largest contour with green.
- Build a square-with-holes and identify outer vs hole using RETR_CCOMP hierarchy.

7. Key Takeaways

- cv2.drawContours(img, list, idx, color, thickness).
- RETR_EXTERNAL default; RETR_TREE for nested.
- Hierarchy is (1, N, 4): Next, Prev, Child, Parent.
- Use hierarchy[0][i][3] == -1 to identify outer contours.

8. What's Next

Moments and centroid — the first shape descriptor.

Lesson 3: Moments & Centroid

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute image / contour moments with `cv2.moments`.
- Use moments to find centroid, area, and orientation.
- Recognise the 7 Hu invariants for shape matching.

Prerequisites

- Module 12 Lesson 2

1. Why This Matters

Moments are weighted sums of pixel positions. They tell you where a shape's centre is, how big it is, and (via Hu invariants) what it looks like — independent of position, rotation, and scale. Used everywhere from blob detection to character recognition.

2. Concept

2.1 Spatial moments

A moment is a sum over pixels:

$$m_{ij} = \sum (x^i \times y^j \times \text{pixel})$$

For a contour, the "pixel" is essentially 1 inside the shape, 0 outside.

Moment	Meaning
m00	area
m10, m01	first moments → use to find centroid
m11, m20, m02	second moments → used for orientation

2.2 OpenCV API

```
M = cv2.moments(contour)      # or cv2.moments(image)
print(M.keys())
```

Returns a dict with all the moments, plus normalised central moments (`nu`) and Hu invariants (`hu`).

2.3 Centroid

```
cx = m10 / m00
cy = m01 / m00
```

```
M = cv2.moments(contour)
cx = int(M["m10"] / M["m00"])
cy = int(M["m01"] / M["m00"])
```

2.4 Hu moments

Seven scalars invariant to translation, rotation, and scale. Useful for matching shapes:

```
hu = cv2.HuMoments(M).ravel() # shape (7,)
```

Often work in log space because the values span many orders of magnitude:

```
import numpy as np
log_hu = -np.sign(hu) * np.log10(np.abs(hu) + 1e-30)
```

2.5 matchShapes for similarity

```
score = cv2.matchShapes(contour_a, contour_b, cv2.CONTOURS_MATCH_I1, 0.0)
```

Lower = more similar. Internally uses Hu moments.

3. Code Examples

Example 1: Centroid of each coin

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)

for c in contours:
    if cv2.contourArea(c) < 100:
        continue
    M = cv2.moments(c)
    cx = int(M["m10"] / M["m00"])
    cy = int(M["m01"] / M["m00"])
    cv2.circle(img, (cx, cy), 4, (0, 0, 255), -1)
cv2.imwrite("coins_centroids.png", img)
```

Expected Output: Coin image with red dots at the centre of each coin.

Example 2: Area from moments vs contourArea

```
import cv2, numpy as np

contour = np.array([[10,10],[100,10],[100,100],[10,100]], dtype=np.int32)
M = cv2.moments(contour)
print("m00 (area):", M["m00"], " contourArea:", cv2.contourArea(contour))
```

Expected Output: Both 8100 (90×90 = 8100). The two functions agree.

Example 3: Hu moments

```
import cv2, numpy as np
```

```

square = np.array([[10,10],[100,10],[100,100],[10,100]], dtype=np.int32)
triangle = np.array([[200,50],[300,150],[100,150]], dtype=np.int32)

for name, c in [("square", square), ("triangle", triangle)]:
    hu = cv2.HuMoments(cv2.moments(c)).ravel()
    print(f"{name}: {hu}")

```

Expected Output: Two arrays of 7 numbers; clearly different — that's why Hu moments are good for shape matching.

Example 4: matchShapes

```

import cv2, numpy as np

square_a = np.array([[10,10],[100,10],[100,100],[10,100]], dtype=np.int32)
square_b = np.array([[200,200],[300,200],[300,300],[200,300]],
dtype=np.int32) # same shape, moved
triangle = np.array([[400,50],[500,150],[300,150]], dtype=np.int32)

print("square vs square:", cv2.matchShapes(square_a, square_b,
cv2.CONTOURS_MATCH_I1, 0.0))
print("square vs tri   :", cv2.matchShapes(square_a, triangle,
cv2.CONTOURS_MATCH_I1, 0.0))

```

Expected Output: First call near 0 (very similar); second much larger (dissimilar).

4. Parameter Sensitivity

Match method	Behaviour
CONTOURS_MATCH_I1	most-used default
CONTOURS_MATCH_I2	different weighting
CONTOURS_MATCH_I3	even different weighting

5. Common Mistakes

Mistake	What Happens	Fix
m00 == 0 (empty contour)	ZeroDivisionError	Filter by area first
Use Hu moments raw (huge range)	Comparisons skewed	Use log magnitudes
Expect Hu to be illumination-invariant	They're scale/rotation/translation only	Pair with proper preprocessing

6. Practice Tasks

- Find each coin's centroid; draw a red dot.
- Compute area via `cv2.moments(c)["m00"]` and compare to `cv2.contourArea`.
- Compare two shapes with `cv2.matchShapes`.

7. Key Takeaways

- `cv2.moments(contour)` returns spatial + central + normalised + Hu moments.
- Centroid: `(m10/m00, m01/m00)`.

- Hu moments are invariant to translation, rotation, scale.
- `cv2.matchShapes` for shape similarity; lower = more similar.

8. What's Next

Bounding boxes and convex hulls — the most common contour summaries.

Lesson 4: Bounding Boxes & Convex Hull

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute axis-aligned and rotated bounding rectangles.
- Compute the convex hull of a contour.
- Approximate a contour with `cv2.approxPolyDP`.

Prerequisites

- Module 12 Lesson 3

1. Why This Matters

Bounding boxes are the universal "where is this object" summary used by detectors, trackers, and downstream code. Hulls and polygon approximations simplify complex contours into something tractable.

2. Concept

2.1 Axis-aligned bounding rect

```
x, y, w, h = cv2.boundingRect(contour)
```

Smallest upright rectangle enclosing the contour. Fast.

2.2 Rotated min-area rect

```
rotrect = cv2.minAreaRect(contour)          # ((cx, cy), (w, h), angle)
box = cv2.boxPoints(rotrect).astype(np.int32)
```

Smallest rectangle of any orientation. Better fit for elongated rotated objects.

2.3 Minimum enclosing circle / ellipse

```
(cx, cy), radius = cv2.minEnclosingCircle(contour)
ellipse = cv2.fitEllipse(contour)           # ((cx, cy), (w, h), angle)
```

2.4 Convex hull

The smallest convex polygon enclosing the contour. "Wrap a rubber band around the points."

```
hull = cv2.convexHull(contour)
# returns_points=True (default) → array of points; False → indices
```

Useful for:

- Simplifying contours.
- Detecting concavities (compare contour area to hull area).

- Hand / shape recognition.

2.5 Polygon approximation

```
epsilon = 0.02 * cv2.arcLength(contour, closed=True)
approx = cv2.approxPolyDP(contour, epsilon, closed=True)
```

Ramer-Douglas-Peucker algorithm — approximates the contour with fewer points. epsilon is the max deviation; typical 1-5% of perimeter.

For a near-rectangular shape, approx will have 4 corners.

2.6 Putting it together

For each contour:

- Get bounding rect → draw bounding box for visualisation.
- Get hull → simplify.
- Get approxPolyDP → detect shape type from vertex count.

3. Code Examples

Example 1: Axis-aligned vs rotated bounding boxes

```
import cv2, numpy as np

# Synthesize a rotated rectangle
img = np.zeros((400, 600), dtype=np.uint8)
rect = ((300, 200), (200, 80), -30) # cx, cy, (w, h), angle
box = cv2.boxPoints(rect).astype(np.int32)
cv2.fillPoly(img, [box], 255)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
c = contours[0]

x, y, w, h = cv2.boundingRect(c)
rot = cv2.minAreaRect(c)
rot_box = cv2.boxPoints(rot).astype(np.int32)

disp = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.rectangle(disp, (x, y), (x + w, y + h), (0, 255, 0), 2)
cv2.drawContours(disp, [rot_box], 0, (0, 0, 255), 2)
cv2.imwrite("bbox_compare.png", disp)
print("axis-aligned area:", w * h, " rotated area:", rot[1][0] * rot[1][1])
```

Expected Output: Green axis-aligned box is big and loose; red rotated box is tight around the rotated rectangle. Console shows the rotated area is much smaller — better fit.

Example 2: Convex hull

```
import cv2, numpy as np

# A star-shaped polygon
pts = []
```

```

for i in range(10):
    angle = i * np.pi / 5
    r = 100 if i % 2 == 0 else 40
    pts.append((int(200 + r * np.cos(angle)), int(200 + r * np.sin(angle))))
pts = np.array(pts, dtype=np.int32)

img = np.zeros((400, 400), dtype=np.uint8)
cv2.fillPoly(img, [pts], 255)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
c = contours[0]
hull = cv2.convexHull(c)

disp = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.drawContours(disp, [hull], 0, (0, 0, 255), 2)
cv2.imwrite("star_hull.png", disp)
print("contour area:", cv2.contourArea(c), " hull area:", cv2.contourArea(hull))

```

Expected Output: Star with red convex hull (outer pentagon). Hull area is larger than contour area — the difference is the "concavity".

Example 3: Polygon approximation — identify shape

```

import cv2, numpy as np

# 4 shapes on a canvas
img = np.zeros((400, 800), dtype=np.uint8)
cv2.rectangle(img, (40, 80), (180, 320), 255, -1) # rect
cv2.fillPoly(img, [np.array([[300, 80], [400, 320], [200, 320]])], 255) # tri
cv2.circle(img, (550, 200), 100, 255, -1) # circle
cv2.fillPoly(img, [np.array([[700, 100], [780, 180], [720, 320], [680, 320]])], 255)
# quad

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in contours:
    eps = 0.02 * cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, eps, True)
    print("vertices:", len(approx))

```

Expected Output: 4 (rect), 3 (triangle), ~6-8 (circle — many vertices), 4 (quad).

Example 4: Min enclosing circle on each coin

```

import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

disp = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
for c in contours:
    if cv2.contourArea(c) < 200: continue
    (cx, cy), r = cv2.minEnclosingCircle(c)

```

```
cv2.circle(dis, (int(cx), int(cy)), int(r), (0, 255, 0), 2)
cv2.imwrite("coins_mincirc.png", dis)
```

Expected Output: Each coin enclosed by a tight green circle.

4. Parameter Sensitivity

Param	Effect
epsilon in approxPolyDP	smaller = closer to original (more vertices)
Hull "returnPoints=True"	gives points; False gives indices

5. Common Mistakes

Mistake	What Happens	Fix
Pass empty contour	Error	Filter by area first
Forget cv2.boxPoints after minAreaRect	TypeError when drawing	Convert to points
approxPolyDP epsilon too small	Hundreds of vertices	Use $0.01-0.05 \times \text{arcLength}$

6. Practice Tasks

- Draw axis-aligned vs rotated bounding box on a rotated rectangle.
- Compute convex hull of a star shape.
- Identify shapes (triangle / square / circle) by approxPolyDP vertex count.

7. Key Takeaways

- cv2.boundingRect (upright) vs cv2.minAreaRect (rotated).
- cv2.minEnclosingCircle for circles.
- cv2.convexHull for outer wrapping.
- cv2.approxPolyDP simplifies contours; useful for shape classification.

8. What's Next

Shape descriptors — aspect ratio, extent, solidity.

Lesson 5: Shape Descriptors

Module: 12 — Contours & Shape Analysis

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Compute aspect ratio, extent, solidity, equivalent diameter.
- Use these to filter contours by shape.
- Combine multiple descriptors for shape classification.

Prerequisites

- Module 12 Lesson 4

1. Why This Matters

A coin and a pencil have similar areas but very different shapes. Aspect ratio, solidity, extent — these small numbers tell you the shape's character. Build them into your contour filter and you can reliably keep only the objects you care about.

2. Concept

2.1 The descriptors

Descriptor	Formula	Tells you
Aspect ratio	w / h of bounding rect	elongation
Extent	contour_area / (w × h)	how rectangular
Solidity	contour_area / hull_area	how convex (1 = convex, < 1 = concave)
Equivalent diameter	$\sqrt{4 \times \text{area} / \pi}$	"diameter if it were a circle"
Orientation	from fitEllipse	angle of major axis

2.2 Typical values

Shape	aspect	extent	solidity
Filled square	1.0	1.0	1.0
Circle	1.0	$\pi/4 \approx 0.785$	1.0
Long pencil	5+	high	1.0
Star	~1	low	low
L-shape	~1	low	low-medium

2.3 Code

```
x, y, w, h = cv2.boundingRect(c)
area = cv2.contourArea(c)
hull_area = cv2.contourArea(cv2.convexHull(c))
```

```
aspect    = w / h
extent    = area / (w * h)
```

```

solidity = area / hull_area if hull_area else 0
eq_diam  = np.sqrt(4 * area / np.pi)

```

2.4 Filter recipes

- Keep round coins: $0.85 < \text{extent} < 0.9$ and $0.95 < \text{solidity}$.
- Keep elongated lines: $\text{aspect} > 3$ or $\text{aspect} < 1/3$.
- Drop concave noise blobs: $\text{solidity} > 0.9$.

2.5 Orientation

```

if len(c) >= 5:
    (cx, cy), (MA, ma), angle = cv2.fitEllipse(c)

```

Angle is in degrees, 0-180. Major axis MA, minor ma.

3. Code Examples

Example 1: Print descriptors for sample shapes

```

import cv2, numpy as np

shapes = []
# Square
sq = np.array([[100, 100], [200, 100], [200, 200], [100, 200]], dtype=np.int32)
shapes.append(("square", sq))
# Circle
img = np.zeros((300, 300), dtype=np.uint8)
cv2.circle(img, (150, 150), 60, 255, -1)
ct, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
shapes.append(("circle", ct[0]))
# L-shape
img = np.zeros((300, 300), dtype=np.uint8)
cv2.fillPoly(img,
[ np.array([[50,50],[200,50],[200,100],[100,100],[100,250],[50,250]]) ], 255)
ct, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
shapes.append(("L", ct[0]))

print(f"{'shape':6s} {'aspect':>8s} {'extent':>8s} {'solidity':>9s}")
for name, c in shapes:
    x, y, w, h = cv2.boundingRect(c)
    area = cv2.contourArea(c)
    hull = cv2.contourArea(cv2.convexHull(c))
    aspect = w / h
    extent = area / (w * h)
    solidity = area / hull if hull else 0
    print(f"{'name':6s} {'aspect':8.3f} {'extent':8.3f} {'solidity':9.3f}")

```

Expected Output:

shape	aspect	extent	solidity
square	1.000	1.000	1.000
circle	1.000	0.785	1.000
L	1.000	~0.39	~0.55

Example 2: Filter by shape

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

round_objects = []
for c in contours:
    area = cv2.contourArea(c)
    if area < 200: continue
    x, y, w, h = cv2.boundingRect(c)
    extent = area / (w * h)
    if 0.7 < extent < 0.85:          # roughly circular
        round_objects.append(c)

print("round objects:", len(round_objects))
```

Expected Output: round objects: ~20 (coins typically have extent around $\pi/4 \approx 0.785$).

Example 3: Orientation via fitEllipse

```
import cv2, numpy as np

# Rotated rectangle
img = np.zeros((300, 600), dtype=np.uint8)
rect = ((300, 150), (200, 50), 30)
box = cv2.boxPoints(rect).astype(np.int32)
cv2.fillPoly(img, [box], 255)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
(cx, cy), (MA, ma), angle = cv2.fitEllipse(contours[0])
print(f"orientation: {angle:.1f}° major={MA:.0f} minor={ma:.0f}")
```

Expected Output: angle close to 30° (the angle we used for the rectangle), MA much larger than ma.

Example 4: Combined classifier

```
import cv2

def classify(c):
    area = cv2.contourArea(c)
    if area < 50: return "noise"
    x, y, w, h = cv2.boundingRect(c)
    aspect = w / h
    extent = area / (w * h)
    if aspect > 3 or aspect < 1/3: return "long-thin"
    if 0.95 < extent <= 1.0:      return "rectangle"
    if 0.7 < extent < 0.85:      return "ellipse / circle"
    return "other"

# pretend we have a contour `c`:
# print(classify(c))
```

4. Parameter Sensitivity

Descriptor	Discriminator
Aspect	elongation
Extent	rectangularity
Solidity	convexity

5. Common Mistakes

Mistake	What Happens	Fix
Divide by zero on small contours	Exception	Filter by area first
Treat aspect > 1 as "wide" only	aspect = w/h depends on orientation	Use min/max of aspect and 1/aspect
Use one descriptor for classification	False positives	Combine 2-3 descriptors

6. Practice Tasks

- Print aspect / extent / solidity for square, circle, L-shape.
- Filter binarised coins() keeping only "circular" extent (~0.785).
- For a rotated rectangle contour, get orientation via fitEllipse.

7. Key Takeaways

- Aspect = w/h; extent = area / bbox; solidity = area / hull.
- Each tells you something different about shape.
- Combine for robust classification.
- Filter contours with descriptor thresholds.

8. What's Next

End of Module 12. Next: Module 13 — Features — template matching, ORB, Hough.

Module 12 — Exercises

15 exercises.

12.1 Finding

- (E) Otsu mask coins(); find contours; print count.
- (M) Filter to area > 500; count.
- (H) Draw all contours on the color image; save.

12.2 Hierarchy

- (E) Draw all contours of coins() in red.
- (M) Fill the largest contour green.
- (H) Build square-with-hole; identify outer vs hole via RETR_CCOMP.

12.3 Moments

- (E) Compute and draw centroids of each coin.
- (M) Verify `m00 == cv2.contourArea`.
- (H) Compare 2 shapes with `cv2.matchShapes`.

12.4 BBox / Hull

- (E) Draw axis-aligned + rotated bbox on rotated rectangle.
- (M) Convex hull of star shape; print areas.
- (H) Classify shapes by `approxPolyDP` vertex count.

12.5 Descriptors

- (E) Print aspect / extent / solidity for square / circle / L-shape.
- (M) Filter coins keeping only extent ~ 0.785 .
- (H) `fitEllipse` orientation on rotated rectangle.

Module 12 — Quiz

10 MCQs.

Q1

cv2.findContours requires: A. color image B. binary mask C. grayscale D. float

Answer: B (L1)

Q2

cv2.RETR_EXTERNAL returns: A. all contours B. only inner contours C. only outermost D. hierarchy info

Answer: C (L2)

Q3

hierarchy[0][i] = [Next, Previous, ?, Parent]. The ? is: A. First_Child B. Last_Child C. Sibling D. Area

Answer: A (L2)

Q4

Centroid is computed as: A. (m10/m00, m01/m00) B. mean of all points C. bounding-box center D. all of these are valid

Answer: A (L3); they happen to coincide for symmetric shapes.

Q5

Hu moments are invariant to: A. translation only B. rotation only C. translation, rotation, scale D. lighting

Answer: C (L3)

Q6

cv2.minAreaRect returns: A. axis-aligned bounding rect B. rotated bounding rect C. circle D. ellipse

Answer: B (L4)

Q7

Convex hull of a star: A. is the star itself B. is the outer pentagon enclosing it C. is empty D. equals the bounding rect

Answer: B (L4)

Q8

cv2.approxPolyDP for a square with epsilon = 2% of perimeter: A. ~50 vertices B. 4 vertices C. 8 vertices D. 1 vertex

Answer: B (L4)

Q9

"Solidity" is: A. area / bbox area B. area / hull area C. $\text{perimeter}^2 / \text{area}$ D. width / height

Answer: B (L5)

Q10

A long thin pencil contour has: A. extent ~ 1, aspect ~ 5 B. extent ~ 0.5, aspect ~ 1 C. solidity 0 D. all-zero descriptors

Answer: A (L5)

Module 12 — Assignment: Coin Sorter

Estimated time: 2 hours

Combines: Module 12 + Modules 9-11

Brief

Take a coin image, segment each coin, and classify by size into 3 categories: small / medium / large. Annotate each with category name and area.

Requirements

- `sort.py INPUT [--output OUT]`
- Threshold → morphological cleanup → contours.
- Filter by area > 200 (drop noise).
- Compute area for each; sort areas; classify by terciles (smallest third / middle / largest).
- Annotate each coin on the original image:
- Draw bounding rectangle.
- Put text: small/med/large + area value.
- Print counts per category.

Constraints

- Modules 1-12 only.
- Use `cv2.findContours`, `cv2.contourArea`.
- Use `argparse`.

Expected Output

```
$ python sort.py datasets/starter/coins.png
small: 6  med: 8  large: 6
saved -> coins_sorted.png
```

`coins_sorted.png` shows boxes + labels per coin.

Grading Rubric

Criterion	Weight
Segmentation + contours	25%
Area-based sort	20%
Annotation correct	25%
Counts printed	10%
Style	20%

Submission

Save as `assignment_module12.py`.

Module 12 — Mini Project: Shape Recogniser

Overview

Take an image with various shapes (triangles, squares, circles, polygons), find each contour, classify by vertex count + descriptors, and annotate.

Concepts Used

- Contours (M12 L1)
- approxPolyDP (M12 L4)
- Shape descriptors (M12 L5)

Features

- shapes.py INPUT
- Otsu threshold; find contours.
- For each contour:
- Compute polygon approximation (epsilon = 2-4% of perimeter).
- Classify by vertex count: 3 = triangle, 4 = square/rect, 5+ = circle / polygon.
- Use extent and solidity to refine (square vs rect, circle vs polygon).
- Annotate label and color outline.

Starter Code

shapes.py:

```
import sys, cv2, numpy as np

def classify(c):
    area = cv2.contourArea(c)
    if area < 100: return None
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.03 * peri, True)
    v = len(approx)
    x, y, w, h = cv2.boundingRect(c)
    aspect = w / h
    extent = area / (w * h)
    if v == 3: return "triangle"
    if v == 4:
        return "square" if 0.85 < aspect < 1.15 else "rectangle"
    if v == 5: return "pentagon"
    if extent > 0.75: return "circle"
    return f"polygon-{v}"

def run(path):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```

out = img.copy()
counts = {}
for c in contours:
    label = classify(c)
    if label is None: continue
    counts[label] = counts.get(label, 0) + 1
    cv2.drawContours(out, [c], 0, (0, 255, 0), 2)
    M = cv2.moments(c)
    if M["m00"]:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])
        cv2.putText(out, label, (cx - 30, cy),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 0, 255), 2)

print(counts)
cv2.imwrite("shapes_classified.png", out)

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/checkerboard.png")

```

Expected Output

For an image containing 1 triangle, 2 squares, 1 circle:

```

{'triangle': 1, 'square': 2, 'circle': 1}
saved -> shapes_classified.png

```

Extension Challenges

- Test on a real "shape sorter toy" photo.
- Add color classification per shape (M4 HSV) — "blue triangle", "red square".
- Use cv2.matchShapes against templates instead of vertex count.

Grading Rubric

Criterion	Weight
Classification rule	30%
Per-shape label drawn	20%
Threshold + segmentation	20%
Counts printed	10%
Style	20%